Meeting 12. Big O and the Machine

Opening The Memory of a Computer · Recap 1 What Big O Counts · Recap 2 What Big O Leaves Out · A deeper look 1 Memory and Computation · A deeper look 2 What Computing Costs · A deeper look 3 Hardware Built for the Product · Closing Back to the O

Meeting 11 computed a layer of neuron scores with `X @ W.T + b` using NumPy and PyTorch on CPUs and GPUs. We now examine how the operation count grows with input size, and how implementation and hardware affect running time. The experiments measure Python overhead, memory access, cache effects, and parallel execution, then use the results to estimate running times.

The examples use group 7's image, *The Bedroom* by Vincent van Gogh, as in the slides. Local measurements were made on a machine without a GPU. T4 measurements come from the class's Meeting 11 posts; published specifications and estimates are identified separately.

Notebooks and reference pages.

Each notebook opens in Google Colab. Choose *Save a copy in Drive* in the *File* menu before you change anything, to save your work.

- The first notebook: before the meeting GPU computation, operation counts, arithmetic rates, memory levels, and timing predictions. This notebook requires a Colab GPU for the GPU experiments.
- The second notebook: rows and columns array layout, traversal order, cache conflicts, padding, and NumPy reductions. No GPU is needed.
- CPU and GPU the two processors, what is inside each of them, and how one product is divided among thousands of cores
- What matrix operations cost the count of every operation of the unit, and the notation O( )

Seven companion slide decks, each available as a web page with explanatory notes and as a PDF:

- The Memory of a Computer the opening (PDF)
- What Big O Counts Recap 1 (PDF)
- What Big O Leaves Out Recap 2 (PDF)
- Memory and Computation A deeper look 1 (PDF)
- What Computing Costs A deeper look 2 (PDF)
- Hardware Built for the Product A deeper look 3 (PDF)
- Back to the O the closing (PDF)

The main points

Each point below links to its explanation and code.

1 A layer uses O(nKm) arithmetic operations

A **neuron** computes a score by multiplying each input coordinate by a weight, summing those products, and adding a bias. A **layer** of K neurons applied to n points with m coordinates computes `X @ W.T + b`. The three-loop implementation executes `total += W[j, c] * X[i, c]` exactly nKm times, with two arithmetic operations per pass.

Doubling n doubles this count; doubling all three dimensions multiplies it by eight. The measured time ratios below are 1.9 and 7.9. Timing ratios vary because throughput can change. **O(nKm)** gives an asymptotic upper bound on operation-count growth, omitting constant factors and lower-order terms. When all three dimensions equal a common size, the count is cubic: doubling that size gives eight times the work. Exponential growth such as 2^{size} instead doubles whenever the size increases by one.

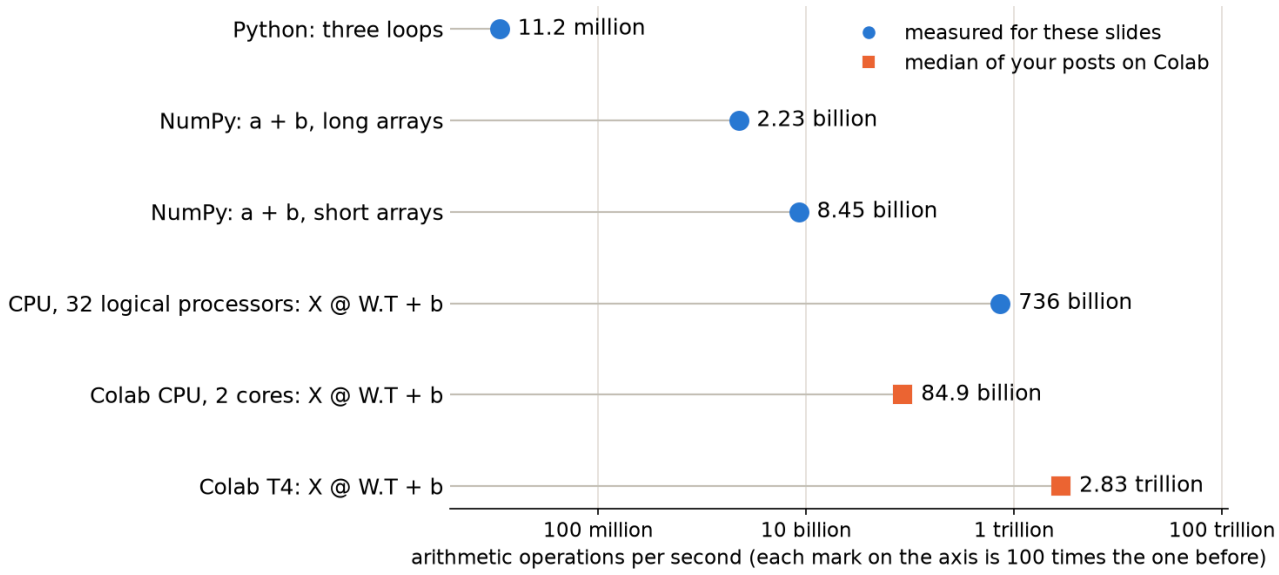
OUTPUT

n	m	K	passes, n x K x m	seconds	ratio to the first	nanoseconds per pass
400	30	20	240,000	0.043	1.0	178
800	30	20	480,000	0.083	1.9	172
800	60	40	1,920,000	0.338	7.9	176

Section 1.3. The page What matrix operations cost lists the count of every operation of the unit.

2 Operations per second and timing estimates

An arithmetic rate counts additions and multiplications per second. The chart compares Python loops at about 11.2 million operations per second, NumPy additions on long and short arrays, and matrix multiplication on three processors: the local CPU with 32 logical processors, Colab's two-core CPU at 84.9 billion operations per second, and the T4 at 2.83 trillion. The Colab values are medians from class posts. A measured rate gives the estimate **time ≈ operations / operations per second**. Its accuracy depends on how well the benchmark represents the proposed computation.



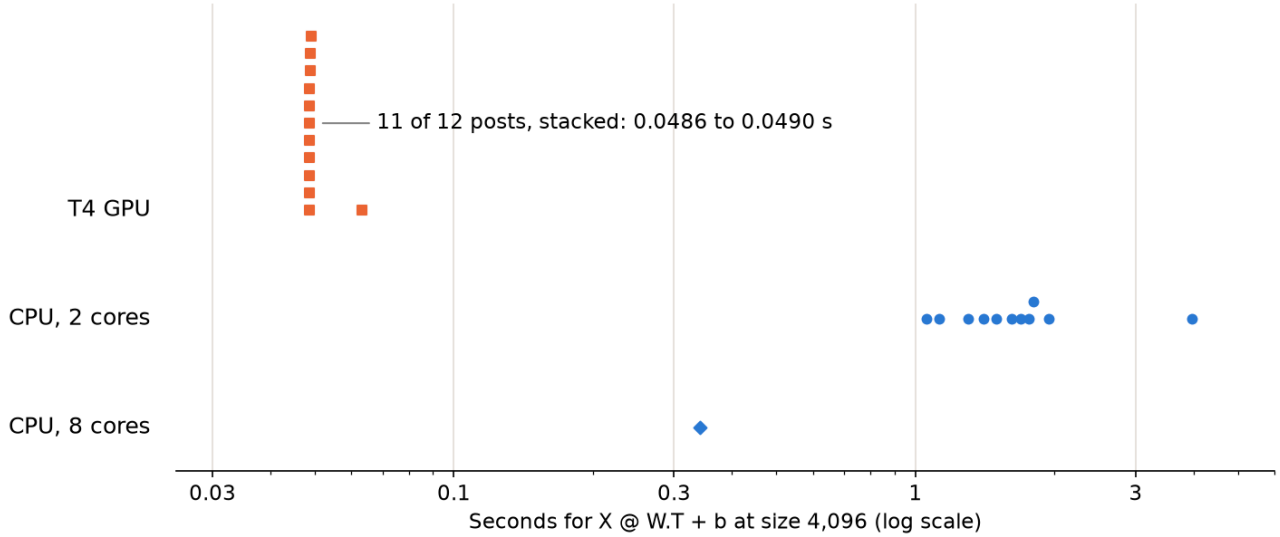
Four local measurements in blue and two medians from class Colab posts in orange.

Sections 1.4 and 1.6.

3 Class measurements on the T4

Twelve groups posted timings for a 4,096 × 4,096 product, approximately 137 billion operations. Eleven T4 times ranged from 0.0486 to 0.0490 seconds; the twelfth was 0.0631 seconds. The two-core CPU times ranged from 1.06 to 1.95 seconds, with one session at 3.97 seconds. The GPU speedup was 22 to 40 in ten posts and 63 in one, with median 33. The session with an eight-core CPU took 0.342 seconds on the CPU and reported a GPU speedup of 7.

Matrix multiplication offers substantial parallel work, which the GPU distributes across its arithmetic units. The CPU and GPU page illustrates this division. Using the GPU also incurs data-transfer costs, discussed below.

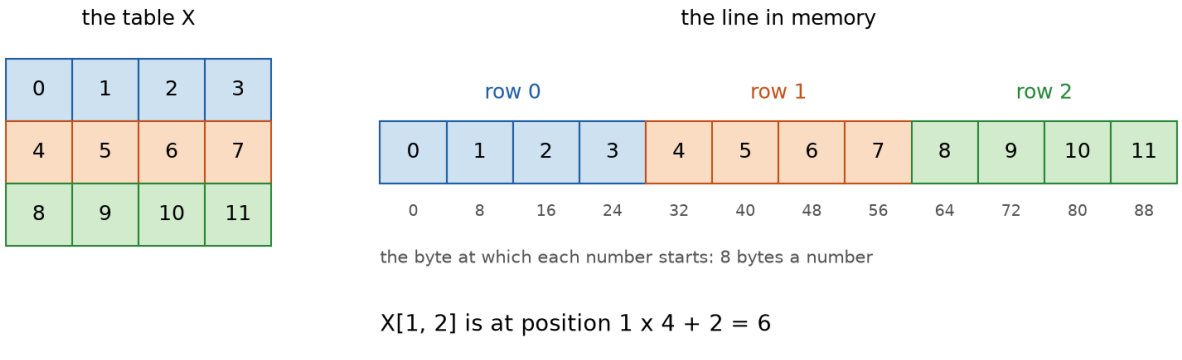


The product at size 4,096, from the twelve posts of the meeting 11 board that measured a T4.

Section 1.2.

4 Matrix storage, strides, and transpose views

A contiguous row-major NumPy matrix stores its rows consecutively. With m columns, entry (i, j) is at element offset $im + j$. The **strides** give the byte offset for a step along each dimension: $8m$ between rows and 8 between columns for float64 entries. `X.T` swaps the dimensions and strides to create a view of the same data. Column-major storage places columns consecutively. Shape and strides describe the layout; the order of memory accesses affects performance.



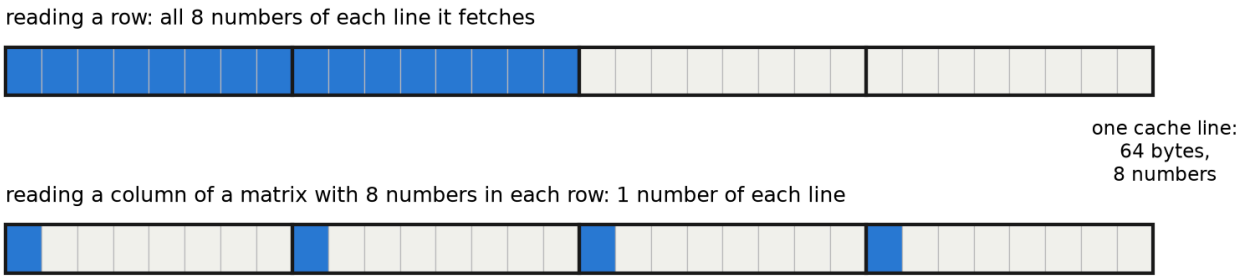
A 3 × 4 matrix stored in row-major order.

Section 2.1.

5 Cache lines and traversal order

A **cache** holds recently accessed data near the processor. Transfers use **cache lines**; a 64-byte line contains eight adjacent float64 entries. Row traversal uses those adjacent entries together and supports prefetching. Column traversal can fetch one entry per line. If the remaining entries are evicted before the next column uses them, they must be read again.

In the local NumPy row and column loops, column traversal took 1.1 times the row-traversal time for a 0.5 MB matrix, and 19.1 times for a 134 MB matrix. Python loops show a smaller difference because interpreter overhead can mask memory-access costs.

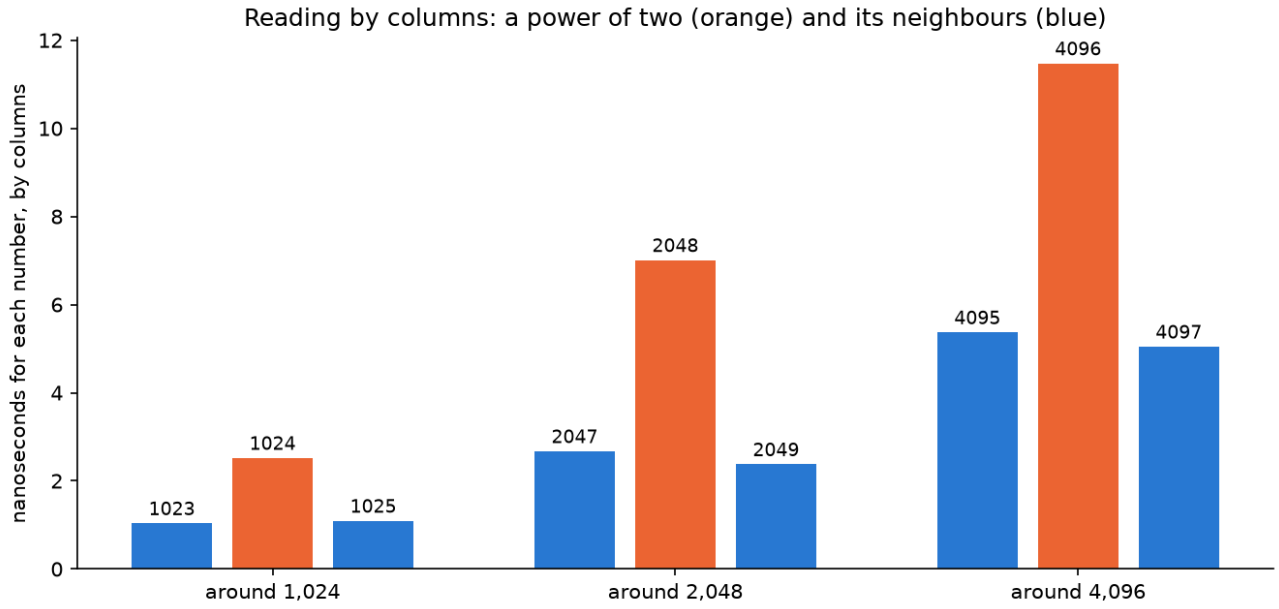


Reading a row uses every number of the lines it fetches; reading a column of a matrix with eight numbers in each row uses one.

Sections 2.2 and 2.3.

6 Cache conflicts and row padding

Each cache line maps to a **set** with limited capacity. In a simple modulo-indexed cache model, certain power-of-two row strides make column accesses compete for a small subset of the sets. A slightly different stride can distribute those accesses more evenly. Locally, a matrix with 2,048 columns took 7.01 nanoseconds per entry to traverse by column, compared with 2.38 for 2,049 columns. **Padding** changes the physical row stride while leaving the logical matrix dimensions unchanged. Section 2.4 explains the model and its limits.



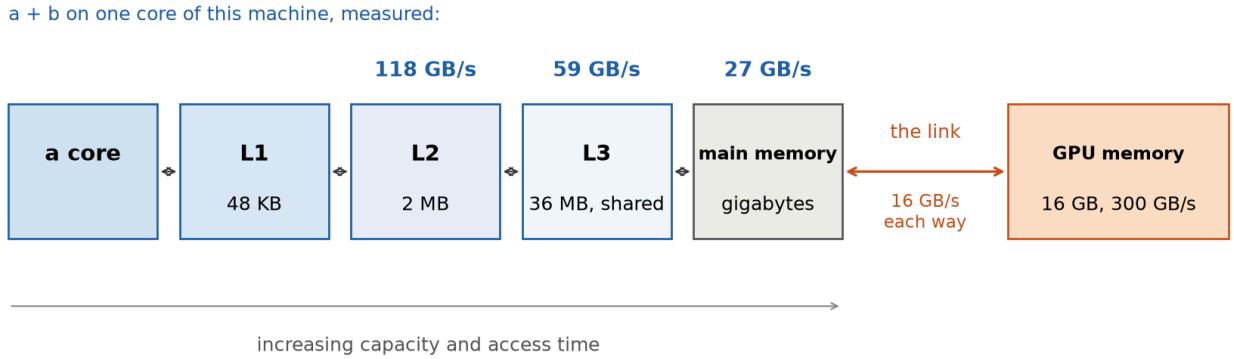
Reading by columns around 1,024, 2,048 and 4,096: the power of two (orange) and its two neighbours (blue), on the machine that built these notes.

Section 2.4.

7 Cache levels and memory bandwidth

The CPU memory hierarchy includes small per-core caches, **L1** and **L2**, a larger shared **L3** cache, and main memory. The array-addition experiment measures how rates change as the data outgrow those levels. Locally, it reached about 9.81 billion additions per second with the arrays in L2, 4.94 billion in L3, and 2.28 billion from main memory.

Each float32 addition reads two 4-byte values and writes one, giving 12 bytes of useful array traffic. Multiplying the rate by 12 estimates **effective bandwidth**: 118 billion bytes per second for L2 and 27.4 billion for main memory in this single-core experiment. The T4's published memory bandwidth is 300 GB/s; its PCIe link provides up to 16 GB/s in each direction.



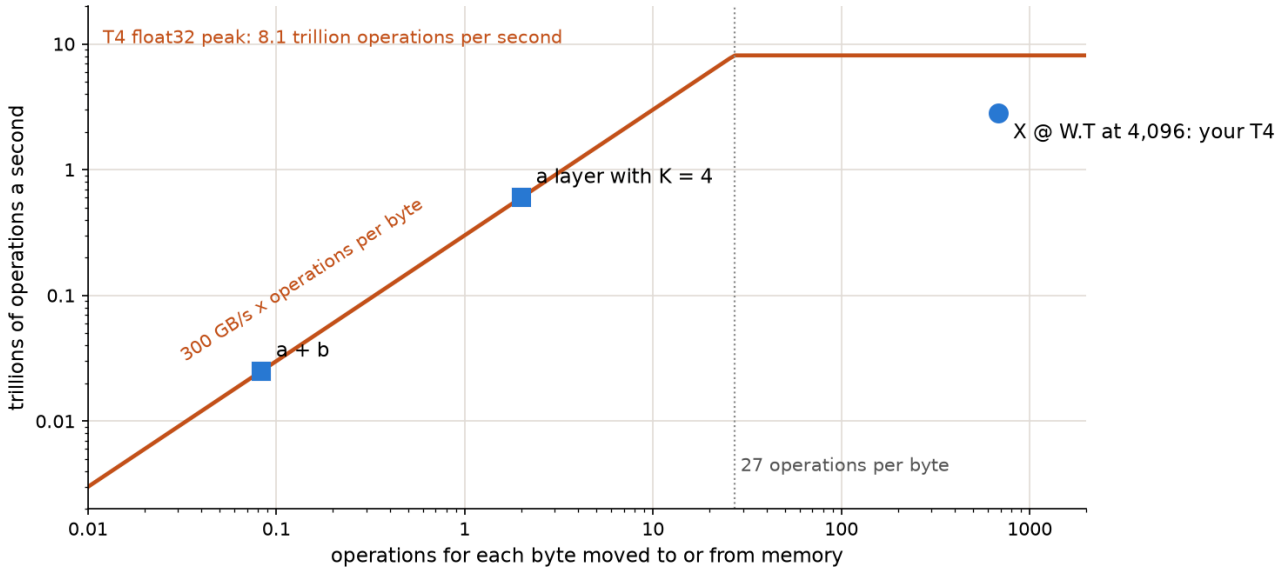
The hierarchy of memories of the machine that built these notes, and the T4's memory across the link; the bandwidths above the boxes were measured with a + b on one core.

Sections 1.5 and 3.1.

8 Arithmetic intensity and the roofline model

Arithmetic intensity is the number of operations per byte transferred. Float32 array addition performs one addition per 12 bytes. At the T4's 300 GB/s memory-bandwidth limit, this permits at most 25 billion additions per second, making the operation **memory-bound** in this model. A square matrix product uses about $2N^3$ operations. Reading its two inputs and writing its output once would transfer $12N^2$ bytes, giving an ideal intensity of $N / 6$. Actual traffic depends on data reuse.

The **roofline** bounds throughput by the smaller of the arithmetic peak and bandwidth times intensity. High-intensity work can be **compute-bound**. The class's 4,096-square product reached about 2.8 trillion operations per second, roughly one third of the T4's float32 peak.



The roofline of the T4, from its data sheet, with a + b, a layer of four neurons and the class's product under it.

Sections 1.4, 3.2 and 3.3.

9 Performance differences beyond the operation count

The table compares implementations, data layouts, and processors. Similar operation counts can produce very different times or rates. The approximate factors come from these notes' measurements and the class posts; compare them with your own notebook results.

What O( ) leaves out	Measured before the meeting	How much
interpreter overhead	Python loops compared with NumPy	about 100 times
the order of reading	a matrix by rows or by columns	1 to about 20 times
the cache's sets	2,048 columns against 2,049	2 to 4 times
memory level	`a + b` in cache compared with main memory	about 5 times
operations per byte, and the cores	`a + b` against the product, on one CPU	hundreds of times
the processor	the T4 against Colab's CPU, in the posts	22 to 40 times

Timing estimates require a measured rate appropriate to the implementation, data size, and processor.

The sections below explain the experiments summarized in this table.

10 What computing costs: joules and dollars

Operation count divided by achieved rate estimates running time. Power in **watts**, or joules per second, multiplied by time estimates **energy**. Hourly rental price multiplied by elapsed hours estimates the processor charge. At the T4's rated 70 W, the measured 4,096-square product gives an energy estimate of 3.4 joules. At the quoted rental price of 35 cents per hour, a trillion operations at the class's measured rate cost about 0.0034 cents.

GPT-3's reported training count was 3.14×10^{23} operations. Patterson and colleagues estimated 1,287 MWh for a run using 10,000 GPUs over about two weeks. Extrapolating the class's rate to that count would take one T4 about 3,526 years. Such estimates depend on achieved throughput, the hardware used, and which costs are included.

Sections 4.1 to 4.5.

Working with the notebooks.

Run each notebook from the top with Shift+Enter. Complete each **Your turn** exercise and run its check. Part 5 of each notebook asks you to have an AI assistant write a function. Record the request in **Your prompt**, specifying the name, arguments, return value, and allowed operations. Answer **Your thinking** in your own words. Keep your measurements so you can compare results across machines.

The first notebook. What Big O counts Open the first notebook

1.1 The layer of meeting 11, on a GPU Part 1

The notebook first checks whether PyTorch can access the Colab GPU. It then applies the layer from Meeting 11 to the pixels of an image: n points, each with red, green, and blue coordinates. Five representative colours are found by grouping pixels around $K = 5$ centres. Each neuron's weight vector is one representative colour; its bias is minus half that vector's squared length. The largest score therefore selects the nearest representative colour. PyTorch computes the scores using float32 **tensors**, its array type, and redraws each pixel in its selected colour.

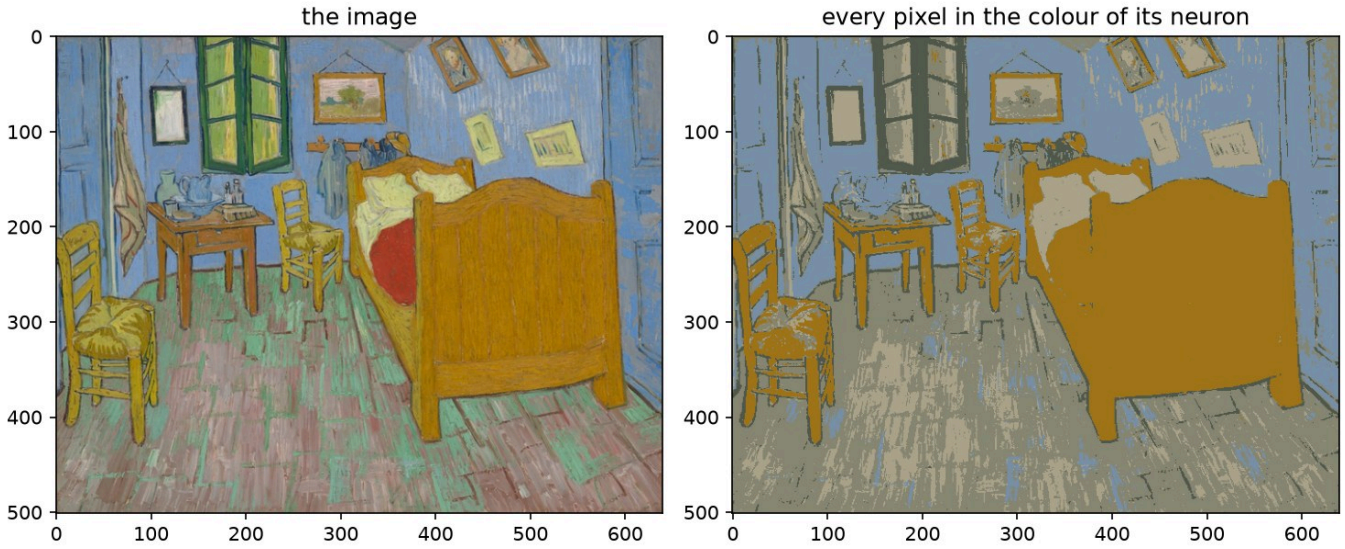


Image 7, and every pixel in the colour of its neuron.

A tensor has a **device**, such as the CPU or a CUDA GPU. `X_t.to(device)` returns a tensor on the requested device, copying data when needed. In this computation, the operands must be on the same device. The exercise asks for `scores_on(device)`, which places the three input tensors on that device and computes the scores. Its check detects a misplaced tensor, missing transpose, or missing bias. The thinking cell asks when transfers are necessary. GPU results remain in GPU memory until copied back; the check uses `.cpu()` to compare them with CPU results.

Code: Part 1, the layer, the function scores_on, and its check cells `part1-layer`, `paste-device`, `part1-check`

```
X = image.reshape(-1, 3)                # one row for each pixel: n points with m = 3 coordinates
n, m = X.shape
K = 5
C = reference_colours(X, K)             # K rows and 3 columns: five colours of the image, darkest first

X_t = torch.from_numpy(X).float()       # the points, as a tensor of float32 numbers
W_t = torch.from_numpy(C).float()       # the weights of the layer: row j is the colour of neuron j
b_t = -0.5 * (W_t ** 2).sum(dim=1)      # the biases: minus half the sum of the squares of each colour
S_t = X_t @ W_t.T + b_t                 # the scores, computed by the CPU: n rows and K columns
print(f"X_t: {tuple(X_t.shape)}   W_t: {tuple(W_t.shape)}   b_t: {tuple(b_t.shape)}   S_t: {tuple(S_t.shape)}   device:
```

```
{S_t.device}")

winner = S_t.argmax(dim=1).numpy()       # for each pixel, the neuron with the largest score
show([image, C[winner].reshape(height, width, 3)], ["the image", "every pixel in the colour of its neuron"])
```

OUTPUT

```
X_t: (320640, 3)   W_t: (5, 3)   b_t: (5,)   S_t: (320640, 5)   device: cpu
```

```
device = "cuda" if GPU else "cpu"        # the GPU, when this session has one


def scores_on(device):
    """Copy the points, the weights and the biases to the device, and compute the scores of the layer there."""
    X_d = X_t.to(device)
    W_d = W_t.to(device)
    b_d = b_t.to(device)
    return X_d @ W_d.T + b_d
```

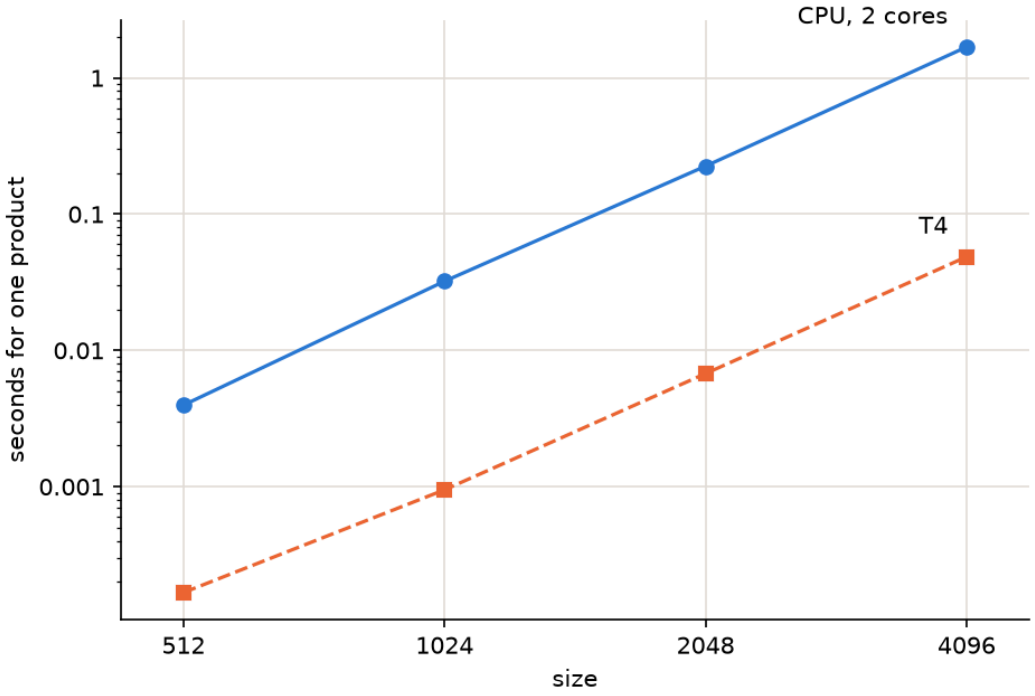
OUTPUT

```
Correct: the scores were computed on the device 'cpu', and .cpu() copies them back. The largest difference from
the CPU's scores: 0.0e+00.
```

1.2 The product on both processors Part 2

The first layer has only five neurons and three input coordinates. Part 2 benchmarks larger products with $n = m = K$ at sizes 512 through 4,096. GPU launches are asynchronous, so `torch.cuda.synchronize()` waits for completion before the timer is read. An unmeasured warm-up operation reduces first-use overhead. The cell also times copying a 4,096-square float32 matrix, about 67 MB, to the GPU. The exercise asks for the GPU speedup at 4,096 and the CPU timing ratio between 2,048 and 4,096. The plot uses logarithmic axes, on which a power law appears as a straight line.

The local table contains only CPU measurements because the build machine has no GPU. Point 3 summarizes the class's T4 results. The plot below shows one group's full table. Doubling the dimension from 2,048 to 4,096 increased CPU time by a factor of 7.5 and T4 time by 7.2, compared with the eightfold increase in the leading operation count. The T4 speedups were 24 at size 512 and 35 at 4,096. Similar growth rates can coexist with substantially different running times.



One group's whole table: Colab's CPU with two cores, and its T4, at the four sizes.

The CPU and GPU page explains how a matrix product can be distributed across many arithmetic units. Transferring its inputs has a separate cost: a 67 MB matrix takes several milliseconds to reach the GPU, compared with roughly fifty milliseconds for the class's product. For smaller computations, transfers can take longer than arithmetic. Section 3.5 estimates those costs from the published link bandwidth.

Code: Part 2, the measurement, the two answers, and the check cells `part2-time`, `part2-chart`, `answers-table`, `part2-check`

```
def finish(device):
    """Wait until the device has finished its work. A GPU may still be working when PyTorch goes on."""
    if device == "cuda":
        torch.cuda.synchronize()


def time_product(size, device):
    """The average time, in seconds, of X @ W.T + b on the device, with n = m = K = size."""
    X = torch.rand(size, size, device=device)        # random float32 numbers, made on the device
    W = torch.rand(size, size, device=device)
    b = torch.rand(size, device=device)
    repeats = max(1, min(200, 8_000_000_000 // size ** 3))     # many repeats when the product is small
    finish(device)
    start = time.perf_counter()
    for _ in range(repeats):
        S = X @ W.T + b
    finish(device)
    return (time.perf_counter() - start) / repeats


sizes = [512, 1024, 2048, 4096]
devices = ["cpu", "cuda"] if GPU else ["cpu"]
```

```
seconds = {}
for d in devices:
    time_product(64, d)                  # one small product that is not measured: the first use of a device is slow
    seconds[d] = [time_product(size, d) for size in sizes]

gpu_name = torch.cuda.get_device_name(0) if GPU else "none"
print(f"X @ W.T + b with n = m = K = size, in seconds. CPU: {os.cpu_count()} logical cores. GPU: {gpu_name}.")
print(f"{'size':>6s}{'multiplications':>18s}{'CPU':>12s}" + (f"{'GPU':>12s}{'CPU / GPU':>12s}" if GPU else ""))
for i, size in enumerate(sizes):
    line = f"{size:6d}{size ** 3:18,d}{seconds['cpu'][i]:12.6f}"
    if GPU:
        line += f"{seconds['cuda'][i]:12.6f}{seconds['cpu'][i] / seconds['cuda'][i]:12.1f}"
    print(line)

if GPU:
    matrix = torch.rand(4096, 4096)                 # one matrix in the memory of the CPU: 4096 x 4096 float32 numbers
    finish("cuda")
    start = time.perf_counter()
    copy = matrix.to("cuda")                        # its copy in the memory of the GPU
    finish("cuda")
    t_copy = time.perf_counter() - start
    megabytes = matrix.element_size() * matrix.nelement() / 1e6
    print(f"\nCopying one matrix of size 4096 ({megabytes:.0f} MB) to the GPU took {t_copy:.4f} seconds; "
          f"the product at that size took the GPU {seconds['cuda'][-1]:.4f} seconds.")
```

OUTPUT

```
X @ W.T + b with n = m = K = size, in seconds. CPU: 32 logical cores. GPU: none.
  size   multiplications         CPU
   512       134,217,728    0.000641
  1024     1,073,741,824    0.003525
  2048     8,589,934,592    0.027090
  4096    68,719,476,736    0.186800
```

OUTPUT

```
Line for the discussion board:
No GPU in this session. CPU with 32 logical cores. Size 4096: 0.187 s. Size 512: 0.6 milliseconds.
```

```
GPU_FACTOR = seconds["cpu"][-1] / seconds["cuda"][-1] if GPU else None
CPU_GROWTH = seconds["cpu"][-1] / seconds["cpu"][-2]
```

OUTPUT

```
Correct: from size 2,048 to 4,096 the time of the CPU grows by a factor of 6.9.
```

1.3 Count the operations Part 3

The three-loop layer implementation executes `total += W[j, c] * X[i, c]` once for every point, neuron, and coordinate: nKm passes, each with a multiplication and an addition. Part 3 times a layer with 400 points, 30 coordinates, and 20 neurons; a layer with twice as many points; and a layer with all three dimensions doubled. The predicted count ratios are 2 and 8. The latter comes from multiplying the three dimension ratios, $2 \times 2 \times 2$.

The measured times approximately follow these counts. The first row takes 178 nanoseconds per pass. At similar throughput, $\text{time} \approx (\text{time per pass}) nKm$. The operation count is $\mathbf{O(nKm)}$; when all three dimensions scale together, it is cubic. Doubling the common size gives eight times the work. By comparison, the exponential 2^{size} doubles each time the size increases by one.

The thinking cell asks about changing a batch and layer: doubling n while halving K preserves nKm, whereas doubling all three dimensions multiplies it by eight. These predict similar time and eightfold time, respectively, only if throughput remains similar.

Code: Part 3, the prediction, its check, and the three loops timed

cells `predict-count`, `part3-check`, `count`

```
FACTOR_N_DOUBLED = 2
FACTOR_ALL_DOUBLED = 8
```

OUTPUT

```
Correct: 240,000, 480,000 and 1,920,000 passes. Now run the next cell and compare the times.
```

```
def layer_loops(X, W, b):
    """The scores of the K neurons of a layer on the n points of X, one multiplication at a time."""
    n, m = X.shape
    K = W.shape[0]
    S = np.empty((n, K))
    for i in range(n):                  # every point
        for j in range(K):              # every neuron
            total = b[j]
            for c in range(m):          # every coordinate
                total += W[j, c] * X[i, c]
            S[i, j] = total
    return S


layers = [(400, 30, 20), (800, 30, 20), (800, 60, 40)]      # n, m and K of the three layers
rng = np.random.default_rng(0)          # a source of random numbers
loop_times = []
print(f"{'n':>6s}{'m':>5s}{'K':>5s}{'passes, n x K x m':>20s}{'seconds':>10s}{'ratio to the first':>21s}{'nanoseconds per pass':>23s}")
for n_, m_, K_ in layers:
    X_, W_, b_ = rng.random((n_, m_)), rng.random((K_, m_)), rng.random(K_)
    start = time.perf_counter()
    layer_loops(X_, W_, b_)
    loop_times.append(time.perf_counter() - start)
    print(f"{n_:6d}{m_:5d}{K_:5d}{n_ * K_ * m_:20,d}{loop_times[-1]:10.3f}{loop_times[-1] / loop_times[0]:21.1f}"
          f"{loop_times[-1] / (n_ * K_ * m_) * 1e9:23.0f}")
```

OUTPUT

n	m	K	passes, n x K x m	seconds	ratio to the first	nanoseconds per pass
400	30	20	240,000	0.043	1.0	178
800	30	20	480,000	0.083	1.9	172
800	60	40	1,920,000	0.338	7.9	176

1.4 Measured operations per second Part 4

Arithmetic throughput is the number of additions and multiplications divided by elapsed seconds. The function `product_rate(size, seconds)` uses the leading count for square multiplication: $2\,\text{size}^3 / \text{seconds}$. The check detects counting only one operation per multiply-add, using a square instead of a cube, or reversing the division.

The local measurements are about 11.2 million operations per second for Python loops, 2.23 billion for NumPy additions on arrays of 16 million entries, and 8.45 billion for the same total number of additions on arrays of about 16 thousand entries. The 4,096-square product reaches 736 billion operations per second, 65,570 times the Python-loop rate. The class's T4 measurements give about 2.8 trillion operations per second. The combined chart appears in point 2. Several implementation and hardware effects explain these differences:

- **Interpreter overhead.** Each Python iteration performs bookkeeping around the arithmetic. NumPy performs array operations in compiled code.
- **Cache capacity.** Short arrays can remain in cache across repeated additions. Long arrays require repeated main-memory transfers. Array addition performs little arithmetic per byte, so memory bandwidth can limit its rate.
- **Data reuse in multiplication.** Each input entry contributes to many products. A blocked implementation reuses entries while they remain in cache, increasing arithmetic per byte transferred. Large products can therefore be compute-bound. See section 3.4.
- **The GPU.** Thousands of small cores work on the same product. The data sheet of the T4 gives 8.1 trillion operations a second at its peak; the class reached a third of that, as section 3.3 discusses.

O( ) describes operation-count growth. The measured rates also depend on how the program is run, where the numbers are, and which processor does the work.

Code: Part 4, the rate of a product, its check, and the five ways of computing cells `paste-rate`, `part4-check`, `rates`

```
def product_rate(size, seconds):
    """Arithmetic operations per second of the product X @ W.T with n = m = K = size, which took `seconds`."""
    return 2 * size ** 3 / seconds
```

OUTPUT

```
Correct: the product at size 4,096 in 0.05 seconds is 2.75 trillion operations per second.
```

```
rng = np.random.default_rng(1)
rates = {}                                # for each way of computing: arithmetic operations per second

# 1. Python, three loops: one multiplication and one addition for each pass
n_, m_, K_ = layers[0]
rates["Python: three loops"] = 2 * n_ * K_ * m_ / loop_times[0]

# 2. NumPy, a + b on long arrays: one addition for each pair of numbers
long = 2 ** 24                            # 16,777,216 numbers in each array
a = rng.random(long, dtype=np.float32)
b = rng.random(long, dtype=np.float32)
c = np.empty(long, dtype=np.float32)
```

```
np.add(a, b, out=c)                       # c = a + b, with the result written into the array c; not measured
start = time.perf_counter()
for _ in range(4):
    np.add(a, b, out=c)
rates["NumPy: a + b, long arrays"] = 4 * long / (time.perf_counter() - start)

# 3. NumPy, a + b on short arrays: the same number of additions
short = 2 ** 14                           # 16,384 numbers in each array
a, b, c = a[:short].copy(), b[:short].copy(), c[:short].copy()
calls = 4 * long // short
np.add(a, b, out=c)
start = time.perf_counter()
for _ in range(calls):
    np.add(a, b, out=c)
rates["NumPy: a + b, short arrays"] = calls * short / (time.perf_counter() - start)

# 4 and 5. The product at size 4,096, from Part 2. The bias adds 4,096 x 4,096 additions, too few to count.
rates["CPU: X @ W.T + b"] = product_rate(4096, seconds["cpu"][-1])
if GPU:
    rates["GPU: X @ W.T + b"] = product_rate(4096, seconds["cuda"][-1])

slowest = min(rates.values())
print(f"{'way of computing':30s}{'operations per second':>24s}{'ratio to the slowest':>23s}")
for name, value in rates.items():
    print(f"{name:30s}{in_words(value):>24s}{value / slowest:23,.0f}")
```

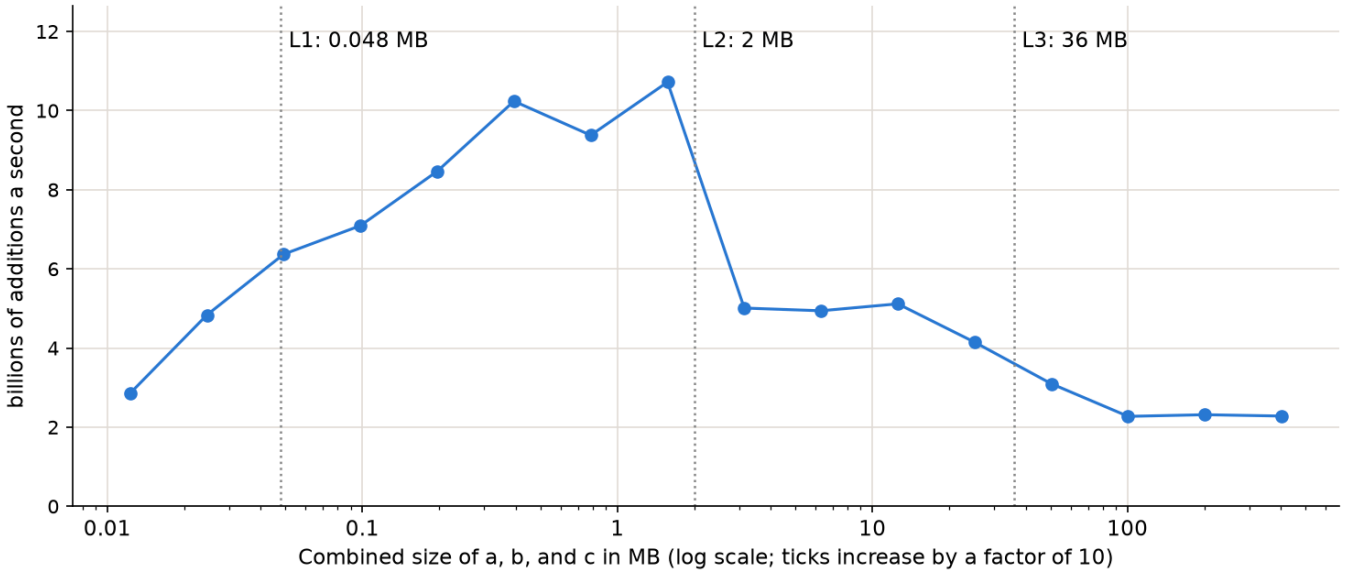
OUTPUT

```
way of computing                operations per second   ratio to the slowest
Python: three loops                        11.2 million                      1
NumPy: a + b, long arrays                  2.23 billion                    199
NumPy: a + b, short arrays                 8.45 billion                    753
CPU: X @ W.T + b                            736 billion                 65,570
```

1.5 Array size and cache capacity Part 5

Part 5 measures `a + b` at sixteen array lengths, from roughly a thousand to 33 million entries, with about 67 million additions at each length. The exercise asks an AI assistant to write `add_rate(length, repeats)`. The function must create two random float32 arrays and an output array, perform an unmeasured warm-up, time `repeats` additions using `time.perf_counter()`, and return `length * repeats / seconds`. It must not loop over individual entries. The check detects returning seconds instead of a rate, omitting the repeat count, and scalar loops.

The chart keeps the best of five runs at each length. The marked cache capacities are 48 KB of L1 and 2 MB of L2 per performance core, and 36 MB of shared L3. Rates generally fall as the arrays outgrow these levels, although transitions are gradual. The notebook reads cache sizes from the system, so Colab plots can have different transition points. For very short arrays, fixed call overhead is a substantial part of the time. Matrix multiplication can continue reusing cached blocks even when the full matrices exceed cache capacity; section 3.4 explains how.



The rate of a + b against the size of the three arrays, on the machine that built these notes; the best of five runs at each length.

Code: Part 5, a function written to the requirements, its check, and the sweep

cells `paste-add-rate`, `part5-check`, `add-sweep`

```
def add_rate(length, repeats):
    """Additions per second of np.add on two arrays of `length` float32 numbers, measured over `repeats` calls."""
    rng = np.random.default_rng()
    a = rng.random(length, dtype=np.float32)
    b = rng.random(length, dtype=np.float32)
    c = np.empty(length, dtype=np.float32)
    np.add(a, b, out=c)                  # once, not measured
    start = time.perf_counter()
    for _ in range(repeats):
        np.add(a, b, out=c)
    seconds = time.perf_counter() - start
    return length * repeats / seconds
```

OUTPUT

```
Correct: add_rate gives 9.84 billion additions a second for two arrays of 16,384 numbers.
```

```
import glob

lengths = [2 ** k for k in range(10, 26)]                # from 1,024 to 33,554,432 numbers in each array
sweep = [add_rate(length, max(1, 2 ** 26 // length)) for length in lengths]      # about 67 million additions each
megabytes = [3 * 4 * length / 1e6 for length in lengths]   # a, b and c: three arrays of numbers of 4 bytes

caches = []                                                 # the caches of this processor, where the system tells them
for folder in sorted(glob.glob("/sys/devices/system/cpu/cpu0/cache/index*")):
    try:
        kind = open(folder + "/type").read().strip()
        level = open(folder + "/level").read().strip()
        size = open(folder + "/size").read().strip()
    except OSError:
        continue
    if kind != "Instruction":
```

```
        factor = {"K": 1e3 * 1.024, "M": 1e6 * 1.048576}.get(size[-1], 1e-6)
        caches.append((f"L{level}", float(size.rstrip("KM")) * factor / 1e6))

print(f"{'numbers in each array':>22s}{'MB of the three arrays':>24s}{'additions per second':>22s}")
for length, mb, rate in zip(lengths, megabytes, sweep):
    print(f"{length:22,d}{mb:24.3f}{in_words(rate):>22s}")
print("Caches of this processor:", ", ".join(f"{name} {mb:g} MB" for name, mb in caches) if caches
      else "the system of this machine does not tell them.")

fig, ax = plt.subplots(figsize=(9, 4.6))
ax.plot(megabytes, [rate / 1e9 for rate in sweep], marker="o", color="#2a78d6")
for name, mb in caches:
    ax.axvline(mb, color="#888888", linestyle=":", linewidth=1)
    ax.annotate(f"{name}: {mb:g} MB", (mb, max(sweep) / 1e9), xytext=(4, -4), textcoords="offset points", fontsize=9,
va="top")
ax.set_xscale("log")
ax.set_xlabel("MB of the three arrays a, b and c (each mark on the axis is 10 times the one before)")
ax.set_ylabel("billions of additions a second")
ax.set_ylim(0, max(sweep) / 1e9 * 1.15)
ax.set_title("The rate of a + b, against the size of the arrays")
ax.grid(True, which="major", color="#e1e0d9", linewidth=0.8)
ax.spines[["top", "right"]].set_visible(False)
fig.tight_layout()
plt.show()

print("Line for the discussion board:")
print(f"a + b: {in_words(max(sweep))} additions a second at best, at {megabytes[sweep.index(max(sweep))]:.2g} MB; "
      f"{in_words(sweep[-1])} at {megabytes[-1]:.0f} MB.")
```

OUTPUT

```
 numbers in each array  MB of the three arrays  additions per second
                 1,024                   0.012          2.46 billion
                 2,048                   0.025          4.16 billion
                 4,096                   0.049          5.16 billion
                 8,192                   0.098          6.94 billion
                16,384                   0.197          8.97 billion
                32,768                   0.393          9.23 billion
                65,536                   0.786          11.3 billion
               131,072                   1.573          8.73 billion
               262,144                   3.146          4.53 billion
               524,288                   6.291          4.38 billion
             1,048,576                  12.583          4.39 billion
             2,097,152                  25.166          4.24 billion
             4,194,304                  50.332          3.08 billion
             8,388,608                 100.663          2.36 billion
            16,777,216                 201.327          2.26 billion
            33,554,432                 402.653          2.19 billion
Caches of this processor: the system of this machine does not tell them.
Line for the discussion board:
a + b: 11.3 billion additions a second at best, at 0.79 MB; 2.19 billion at 403 MB.
```

1.6 Predict the time of a layer, then measure it Part 6

Using **time ≈ operations / operations per second**, a layer of 1,000 neurons on 60,000 images with 784 coordinates requires about 94 billion operations. The measured rates predict 2.33 hours for Python loops, 1.11 seconds for Colab's CPU, and 33.3 milliseconds for its T4. Each group chooses a layer below a trillion

operations and 2 GB of input and output arrays, predicts its time, and measures CPU and GPU execution. The slow Python implementation is estimated only.

The local example uses 50,000 colour images of 32 × 32 pixels, or 3,072 coordinates, and 512 neurons: 157 billion operations. The predicted CPU time is 214 milliseconds; the measured time is 224 milliseconds, a ratio of 1.05. Differences can arise because the benchmark used square matrices, because of first-use overhead, or because the machine was doing other work.

Arithmetic intensity also affects a prediction. With only four neurons, a float32 layer performs about two operations per input byte, before counting other traffic. Memory bandwidth can then limit throughput, making a rate measured on a large square product unsuitable. The final thinking cell asks about this case; section 3.2 places it on the roofline.

Code: Part 6, the layer of these notes, the prediction and the measurement, and the predictions for the layer of 94 billion operations cells `choose-layer`, `layer-predict`, `recap-predict`

```
n_layer, m_layer, K_layer = 50_000, 3_072, 512      # 50,000 colour images of 32 by 32 pixels, a layer of 512 neurons
```

OUTPUT

```
A layer of K = 512 neurons on n = 50,000 points with m = 3,072 coordinates: 157 billion operations, 0.72 GB of
numbers.
way of computing                 predicted            measured   measured / predicted
Python: three loops             3.89 hours             not run
CPU                        214 milliseconds    224 milliseconds                   1.05

Line for the discussion board:
Layer n = 50,000, m = 3,072, K = 512: 157 billion operations. Python: three loops: predicted 3.89 hours; CPU:
predicted 214 milliseconds, measured 224 milliseconds.
```

OUTPUT

```
K = 1,000 neurons, n = 60,000 images of 28 x 28 pixels (m = 784): 94.1 billion operations
implementation                        predicted time
Python: three loops                       2.33 hours
CPU, 32 logical processors: X @ W.T + b  128 milliseconds
Colab CPU, 2 cores: X @ W.T + b         1.11 seconds
Colab T4: X @ W.T + b              33.3 milliseconds
```

The class measurements also illustrate the limits of throughput estimates. One group's 583-billion-operation layer was predicted to take 4.6 seconds on the CPU and took 6.7. Its T4 prediction was 206 milliseconds, while the measured time was 160 milliseconds. The predictions were within a factor of two, rather than exact.

The second notebook. What Big O leaves out Open the second notebook

2.1 Matrix layout and strides Part 1

Every byte has a memory **address**. The contiguous row-major matrix here stores rows consecutively. For a 3 × 4 float64 matrix, `X.ravel()` lists entries in their storage order and `X.strides` returns `(32, 8)` : 32 bytes between rows and 8 between columns. With m columns, entry (i, j) has element offset $im + j$. The exercise implements this as `position(i, j, m)` and checks it on a rectangular matrix. Column-major storage instead uses $jn + i$ for a matrix with n rows.

`X.T` creates a transpose view by swapping dimensions and strides. It shares the original data, as `np.shares_memory` confirms. Fortran, MATLAB, and R commonly use column-major storage. `np.asfortranarray(X)` creates that layout for this matrix, with strides `(8, 24)` .

Code: Part 1, the line, the function position, its check, and the transpose cells `one-line` , `paste-position` , `part1-check` , `strides`

```
X = np.arange(12.0).reshape(3, 4)        # the numbers 0 to 11, as 3 rows of 4
print(X)
print("the line in memory:", X.ravel())
print("bytes for one number:", X.itemsize, "  strides:", X.strides)
```

OUTPUT

```
[[ 0.  1.  2.  3.]
 [ 4.  5.  6.  7.]
 [ 8.  9. 10. 11.]]
the line in memory: [ 0.  1.  2.  3.  4.  5.  6.  7.  8.  9. 10. 11.]
bytes for one number: 8   strides: (32, 8)
```

```
def position(i, j, m):
    """The position, in the line of memory, of the number in row i and column j of a matrix with m columns."""
    return i * m + j
```

OUTPUT

```
Correct: every number of a matrix with 5 rows and 7 columns is where position says.
```

```
XT = X.T
print("X.T has shape", XT.shape, "and strides", XT.strides, "  same memory as X:", np.shares_memory(X, XT))
C = np.asfortranarray(X)                 # the same table, stored one column after another
print("in column order, the line is:", C.ravel(order="K"), "  strides:", C.strides)
```

OUTPUT

```
X.T has shape (4, 3) and strides (8, 32)   same memory as X: True
in column order, the line is: [ 0.  4.  8.  1.  5.  9.  2.  6. 10.  3.  7. 11.]   strides: (8, 24)
```

2.2 Loops in Python, by row and by column Part 2

The two Python functions sum every matrix entry in row order and column order. On the local 2,000 × 2,000 float64 matrix, row traversal takes 91 nanoseconds per entry and column traversal takes 93, a ratio of 1.02. A separate run gave a ratio of 1.05. Interpreter overhead helps explain why the difference is relatively small: each scalar access requires Python and NumPy bookkeeping in addition to reading memory. Part 3 reduces that overhead by processing a complete row or column per NumPy call.

Code: Part 2, the two loops, timed cell `python-loops`

```
def sum_by_rows(X):
    """Every number of X, row after row: the order of the line."""
    n, m = X.shape
    total = 0.0
    for i in range(n):
        for j in range(m):
            total += X[i, j]
    return total


def sum_by_columns(X):
    """Every number of X, column after column: a jump of a whole row at each step."""
    n, m = X.shape
    total = 0.0
    for j in range(m):
        for i in range(n):
            total += X[i, j]
    return total


n = 2000
X = rng.random((n, n))
t_rows = best_time(lambda: sum_by_rows(X), repeats=2)
t_columns = best_time(lambda: sum_by_columns(X), repeats=2)
print(f"{n} x {n} numbers, {X.nbytes / 1e6:.0f} MB")
print(f"by rows:    {t_rows:.3f} s, {t_rows / n ** 2 * 1e9:.0f} nanoseconds for each number")
print(f"by columns: {t_columns:.3f} s, {t_columns / n ** 2 * 1e9:.0f} nanoseconds for each number")
print(f"by columns / by rows: {t_columns / t_rows:.2f}")
```

OUTPUT

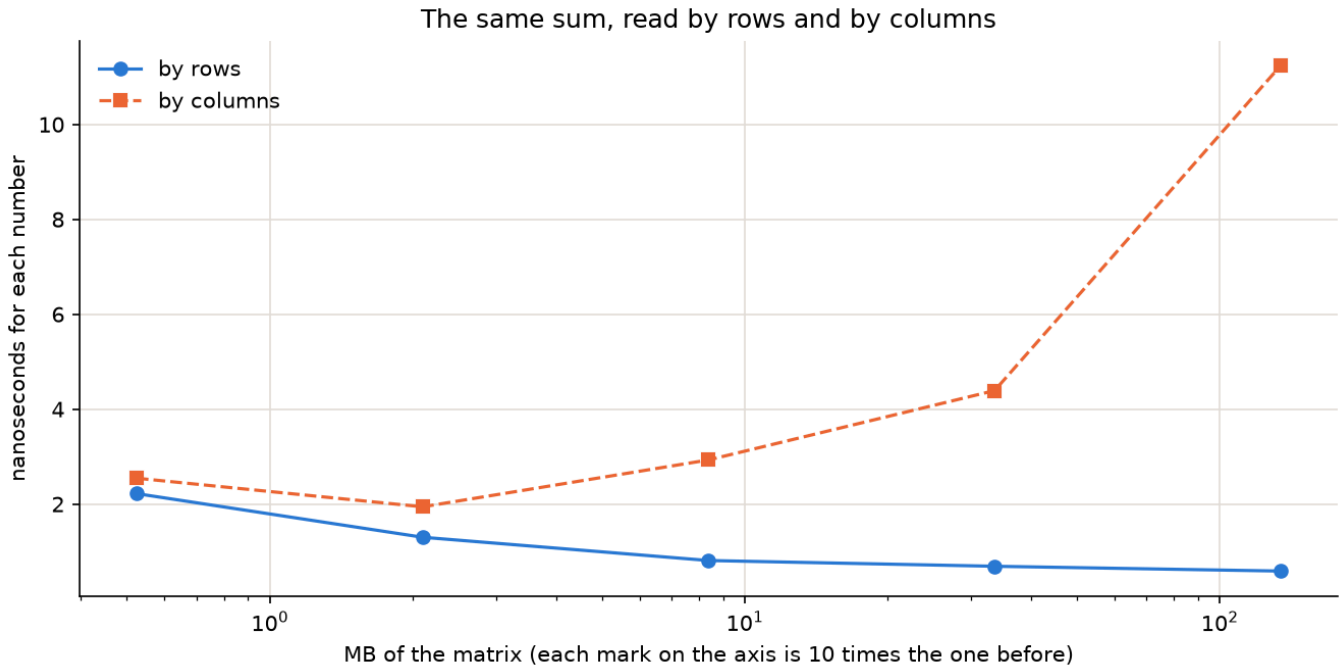
```
2000 x 2000 numbers, 32 MB
by rows:    0.365 s, 91 nanoseconds for each number
by columns: 0.372 s, 93 nanoseconds for each number
by columns / by rows: 1.02
```

2.3 Rows and columns, with NumPy inside the loop Part 3

`sum_by_rows_numpy` retains one Python loop and passes each row, `X[i, :]`, to NumPy for summation. The exercise asks for the corresponding function using columns, `X[:, j]`. Its check examines the executable syntax for a loop and column access. The benchmark compares the functions on matrices from 256 × 256, about 0.5 MB, to 4,096 × 4,096, about 134 MB.

The two traversal orders have similar costs for matrices that fit in the local 2 MB L2 cache. Column traversal takes 3.6 times as long at 8 MB, 6.4 at 34 MB, and 19.1 at 134 MB. The largest matrix also exceeds the 36 MB L3 cache. For the smallest matrices, NumPy call overhead remains significant.

A 64-byte cache line contains eight adjacent float64 entries. Row traversal uses these together. Column traversal uses one entry and advances by the row stride; other entries may be evicted before the next column reaches them. This explains a source of the observed difference. Cache conflicts, prefetching, and other overhead also affect the timings. The notebook marks the cache capacities of the machine on which it runs.



Row and column traversal timings across matrix sizes, measured locally.

Code: Part 3, the two functions, the check, and the measurement

cells `paste-columns`, `part3-check`, `rows-columns`

```
def sum_by_rows_numpy(X):
    """Every number of X: a loop over the rows, and NumPy adds up each row."""
    total = 0.0
    for i in range(X.shape[0]):
        total += X[i, :].sum()
    return total


def sum_by_columns_numpy(X):
    """Every number of X: a loop over the columns, and NumPy adds up each column."""
    total = 0.0
    for j in range(X.shape[1]):
        total += X[:, j].sum()
    return total
```

OUTPUT

```
Correct: sum_by_columns_numpy adds up every number, one column at a time.
```

```
sizes = [256, 512, 1024, 2048, 4096]
by_rows, by_columns, megabytes = [], [], []
for n in sizes:
    X = rng.random((n, n))
    megabytes.append(X.nbytes / 1e6)
    by_rows.append(best_time(lambda: sum_by_rows_numpy(X)) / n ** 2 * 1e9)
    by_columns.append(best_time(lambda: sum_by_columns_numpy(X)) / n ** 2 * 1e9)
del X

print(f"{'size':>6s}{'MB':>8s}{'by rows, ns a number':>23s}{'by columns':>13s}{'columns / rows':>17s}")
for n, mb, r, c in zip(sizes, megabytes, by_rows, by_columns):
    print(f"{n:6d}{mb:8.1f}{r:23.2f}{c:13.2f}{c / r:17.1f}")
```

OUTPUT

size	MB	by rows, ns a number	by columns	columns / rows
256	0.5	2.22	2.55	1.1
512	2.1	1.30	1.94	1.5
1024	8.4	0.81	2.93	3.6
2048	33.6	0.69	4.39	6.4
4096	134.2	0.59	11.24	19.1

2.4 Power-of-two strides and padding Part 4

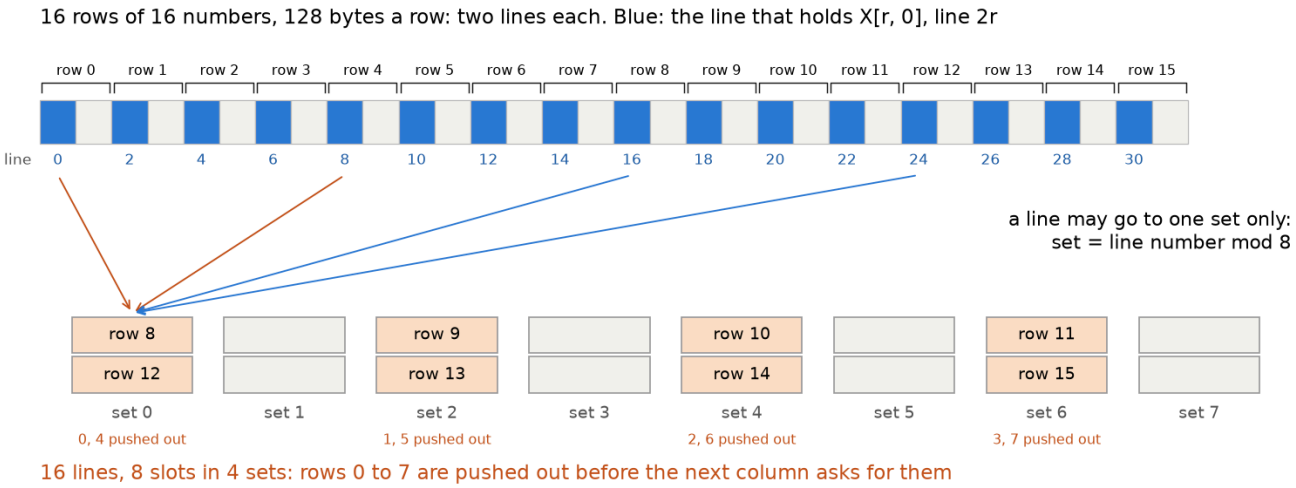
The experiment compares column traversal at widths 1,024, 2,048, and 4,096 and at the adjacent widths. It asks whether a slightly larger matrix must cost more per entry to read. Locally, width 2,048 takes 7.01 nanoseconds per entry, compared with 2.67 and 2.38 at widths 2,047 and 2,049. The plot appears in point 6.

A cache is divided into **sets**; the address of a line decides which set it goes to, and each set holds a limited number of lines. In a simple modulo-indexed model, consecutive lines go to consecutive sets. A large power-of-two row stride can instead send a column's lines to a small fraction of the sets. Those sets can fill even when others have unused space, causing lines to be evicted before the next column needs them. Padding changes the stride and can reduce these conflicts.

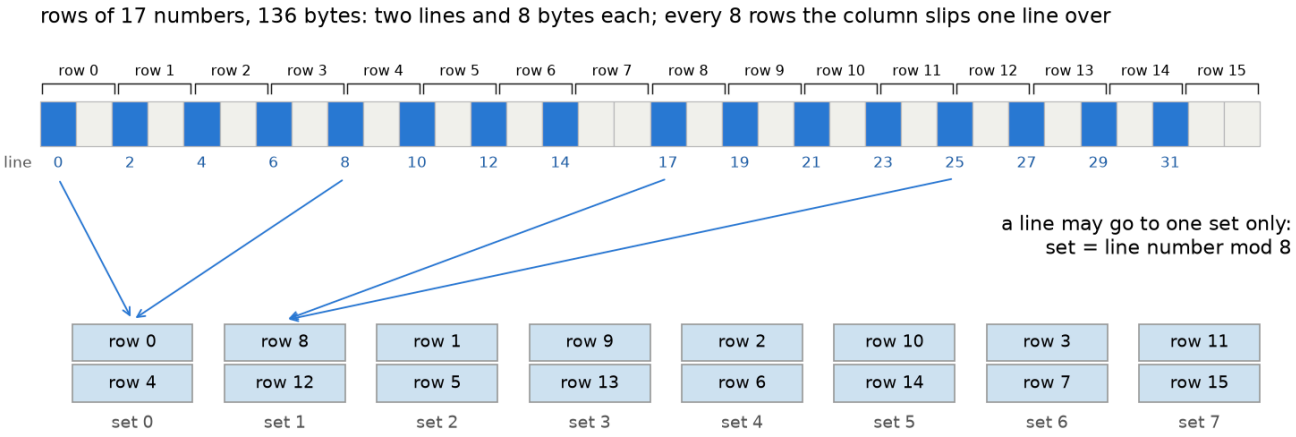
The drawings use this model with 64-byte lines, eight `float64` numbers per line, a base address of zero, an initially empty cache, and least-recently-used (LRU) replacement. They show the state after reading column 0 in row order, with no prefetching or other accesses. For 16 rows of 16 numbers, the first number of row r lies in line $2r$. With 8 sets of 2 lines, its set is $2r \bmod 8$: only sets 0, 2, 4 and 6 are used. Sixteen lines compete for eight slots, and rows 0 to 7 have been evicted by the end of column 0. For rows of 17 numbers, the first-column line is $2r + \lfloor r / 8 \rfloor$. Rows 0 to 7 use the even sets twice each, and rows 8 to 15 use the odd sets twice each. All sixteen lines fit at the end of column 0. This does not guarantee a hit on every read of column 1: rows 7 and 15 cross into new lines. With least-recently-used (LRU) replacement, row 7 also evicts the line needed by row 11, giving three misses in column 1.

For a larger example, use the capacities of this laptop's performance core: the L1 data cache is 48 KiB with 12 lines per set, so it has $48 \cdot 1024 \,/\, (64 \cdot 12) = 64$ sets; L2 is 2 MiB with 16 lines per set, so it has $2 \cdot 1024^2 \,/\, (64 \cdot 16) = 2048$ sets. In the same aligned model, a row of 2,048 numbers is 256 lines, and $256r \bmod 2048$ takes eight values. For 2,048 rows, 256 lines compete for each set's 16 slots, 128 slots in total. With 2,049 numbers per row, the first-column line is $256r + \lfloor r / 8 \rfloor$. Write $r = 8q + t$, where $0 \le q < 256$ and $0 \le t < 8$. Its set is $256t + q$, which visits each of the 2,048 sets once.

That exact count applies to column 0 and a line-aligned base in this model. The actual L2 placement depends on physical address mapping, which a contiguous NumPy array does not establish. Alignment, prefetching, replacement, and other cache levels also affect the measured result. The model explains a source of conflicts; its set counts alone do not predict a timing ratio.



Rows of 16 numbers, two lines each: the line that holds the first number of row r is line 2r, and it may go only to set 2r mod 8. Sixteen lines, eight slots.



Rows of 17 numbers, two lines and 8 bytes each: every 8 rows the column slips one line over, into an odd set. After column 0, all sixteen fetched lines fit, two per set.

Padding can reduce these conflicts by storing each logical row with an unused entry at its end. For example, `np.zeros((n, m + 1))[:, :m]` has shape (n, m) but a physical row stride of $m + 1$ entries. The exercise implements `padded(X)`. For a 2,048-square matrix, the local check measures 26.6 milliseconds without padding and 11.5 with it, a factor of 2.3. Matrix libraries may use padding internally.

Code: Part 4, the measurement around three powers of two, the function padded, and its check cells `pow2`, `paste-padded`, `part-pow2-check`

```
groups = [1024, 2048, 4096]
sizes_pow2 = [n for k in groups for n in (k - 1, k, k + 1)]
pow2_ns = {n: float("inf") for n in sizes_pow2}
for k in groups:
    matrices = {n: rng.random((n, n)) for n in (k - 1, k, k + 1)}
    for _ in range(5):                  # five rounds over the three sizes: the best time of each is kept
        for n, X in matrices.items():
            start = time.perf_counter()
            sum_by_columns_numpy(X)
            pow2_ns[n] = min(pow2_ns[n], (time.perf_counter() - start) / n ** 2 * 1e9)
del matrices, X

print(f"{'size':>6s}{'by columns, ns a number':>26s}")
for n in sizes_pow2:
```

OUTPUT

```
  size   by columns, ns a number
  1023                      1.04
  1024                      2.52   <- a power of two
  1025                      1.09
  2047                      2.67
  2048                      7.01   <- a power of two
  2049                      2.38
  4095                      5.37
  4096                     11.47   <- a power of two
  4097                      5.05
```

```
def padded(X):
    """A copy of X stored in rows one number longer than X's: the same table, with one unused number after each row."""
    n, m = X.shape
    P = np.zeros((n, m + 1))
    P = P[:, :m]
    P[:] = X
    return P
```

OUTPUT

```
Correct: padded keeps the table and stores each row one number longer.

The columns of a matrix of 2,048 by 2,048 numbers:
stored as it is, rows of 2,048 numbers:     26.6 milliseconds
padded, rows of 2,049 numbers:            11.5 milliseconds
as it is / padded: 2.3
```

2.5 Column sums, read by rows Part 5

Column sums can be accumulated while reading a matrix by rows. Start with m zeros, one sum per column, and add each complete row to them. The exercise asks an AI assistant for `column_sums_by_rows(X)`, using one row loop and vector addition. It forbids column loops, `X[:, j]`, and calls to `X.sum` or `np.sum`. The check examines executable statements, so mentioning those methods in a docstring is allowed.

On the 134 MB matrix, the local column loop takes 191.4 milliseconds, the row-accumulation loop 6.7, and NumPy's `X.sum(axis=0)` 4.3. NumPy can choose an efficient traversal independently of the axis being reduced.

Code: Part 5, a function written to the requirements, and its check with the timing cells `paste-column-sums`, `part4-check`

```
def column_sums_by_rows(X):
    """The sum of each column of X, read row by row: X.sum(axis=0) without NumPy's sum."""
    total = np.zeros(X.shape[1])
    for i in range(X.shape[0]):
        total += X[i]
    return total
```

OUTPUT

```
Correct: column_sums_by_rows returns the sum of every column, reading the matrix by rows.

The sums of the 4,096 columns of a matrix of 134 MB:
a loop over the columns, X[:, j].sum():      191.4 milliseconds
column_sums_by_rows, a loop over the rows:      6.7 milliseconds
NumPy's own X.sum(axis=0):                      4.3 milliseconds
```

2.6 NumPy traversal and mixed layouts Part 6

NumPy can choose an efficient memory traversal for some operations. For the 4,096-square matrix, the ratio of column-sum time to row-sum time is 0.6. Column sums do not require an explicit loop down each column. Matrix-vector multiplication with column-major storage takes 1.4 times the row-major time.

Other operations must accommodate different input and output layouts. Here, `X.T.copy()` takes 10.0 times as long as `X.copy()`, and `X + X.T` takes 8.3 times as long as `X + X`. Each pair has the same number of copied entries or additions, but different memory-access patterns.

Code: Part 6, four pairs of operations cell `numpy-pairs`

```
n = 4096
X = rng.random((n, n))
F = np.asfortranarray(X)                # the same matrix, stored in column order
v = rng.random(n)
pairs = [("X.sum(axis=1)", lambda: X.sum(axis=1), "X.sum(axis=0)", lambda: X.sum(axis=0)),
         ("X @ v", lambda: X @ v, "F @ v, F in column order", lambda: F @ v),
         ("X.copy()", lambda: X.copy(), "X.T.copy()", lambda: X.T.copy()),
         ("X + X", lambda: X + X, "X + X.T", lambda: X + X.T)]
print(f"{'operation':16s}{'ms':>8s}    {'the same numbers, another order':32s}{'ms':>8s}{'ratio':>8s}")
for name_a, a, name_b, b in pairs:
    ta, tb = best_time(a), best_time(b)
    print(f"{name_a:16s}{ta * 1e3:8.1f}    {name_b:32s}{tb * 1e3:8.1f}{tb / ta:8.1f}")
```

OUTPUT

operation	ms	the same numbers, another order	ms	ratio
X.sum(axis=1)	7.3	X.sum(axis=0)	4.4	0.6
X @ v	3.0	F @ v, F in column order	4.3	1.4
X.copy()	20.6	X.T.copy()	205.5	10.0
X + X	24.7	X + X.T	205.1	8.3

Image layouts and strides.

An image of h rows and w columns of pixels with three colours has shape $(h, w, 3)$ and is stored pixel after pixel: the red, green and blue numbers of one pixel sit side by side, and the red channel `image[:, :, 0]` is every third number of the line. PyTorch image layers commonly use the logical shape $(N, 3, h, w)$. A contiguous array in that order stores each channel as a block, but strides determine the physical layout. Permuting dimensions can create a view without moving data.

Memory bandwidth and computation

Cache capacity and bandwidth affect how quickly a processor can obtain data. Arithmetic intensity describes how much computation it performs per byte transferred. The following sections use these quantities to explain the roofline model, blocked multiplication, and GPU-transfer costs. T4 specifications come from NVIDIA's data sheet; CPU measurements come from the local notebook run.

3.1 Bytes a second at each level

The sweep of Part 5, run again for these notes with the best of five runs kept, samples three size ranges: the three arrays fitting in L2, fitting in L3, and exceeding both caches. Each addition reads two numbers of 4 bytes and writes one, 12 bytes in all, so the rate of additions in each range estimates the **effective bandwidth** of that level, the bytes it delivers in a second to one core, since `np.add` uses one core. The table uses the median rate within each size range. Its byte counts describe useful array traffic; hardware transfers can also include write allocation and other overhead.

OUTPUT

the three arrays	capacity	additions a second	bytes a second
in L2	2 MB, one core	9.81 billion	118 billion
in L3	36 MB, shared	4.94 billion	59.3 billion
in main memory	gigabytes	2.28 billion	27.4 billion

The drawing of idea 7 puts these numbers over the hierarchy: a core, its L1 of 48 kilobytes and its L2 of 2 megabytes, the L3 of 36 megabytes shared by all the cores, the main memory of gigabytes, and, across the link, the GPU's memory, 16 gigabytes at 300 gigabytes a second by the T4's data sheet. The link is PCI Express, 32 gigabytes a second in both directions together, 16 each way. The CPU's memory hierarchy trades access speed for capacity. `a + b` is memory-bound at every level: it does too little arithmetic for each byte to keep the core busy, and its rate is the bandwidth divided by 12.

3.2 Operations per byte, and the roofline

Arithmetic intensity is the operation count divided by the bytes transferred to and from a specified memory level. Together with that level's bandwidth and the processor's arithmetic peak, it determines the roofline bound.

$$a + b\text{: } \frac{1 \text{ operation}}{12 \text{ bytes}}, \qquad XW^T \text{ at size } N\text{: } \frac{2N^3 \text{ operations}}{3 \times 4N^2 \text{ bytes}} = \frac{N}{6}.$$

Multiplying two matrices of size N takes $2N^3$ operations; reading both and writing the result once moves $12N^2$ bytes; so the product does $N / 6$ operations for each byte, 683 at $N = 4,096$, under the idealized assumption that each number is fetched once and reused N times. Section 3.4 explains how blocking reduces repeated memory reads. The **roofline** draws both limits of a processor on one chart, with operations per byte along the bottom and operations a second up the side: a sloping line, the bandwidth times the operations per byte, and a flat line, the peak rate of the arithmetic. A computation can go no faster than the lower of the two,

$$\text{speed} \le \min\left(\text{peak rate}, \text{bandwidth} \times \text{operations per byte}\right),$$

and the two lines meet at the **ridge**: for the T4, 8.1 trillion operations a second divided by 300 gigabytes a second, 27 operations per byte. Left of the ridge the memory sets the speed; right of it the arithmetic does. The chart of idea 8 places three computations under the T4's roof. `a + b`, at one operation for 12 bytes, can reach at most 25 billion additions a second on the T4, under this bandwidth limit. A layer with $K = 4$ neurons does about two operations for each byte of `X` it reads and is memory-bound even on a GPU, which is why a prediction from the rate of a large product would be far too optimistic for it, the question of Part 6. The product at size 4,096 is far to the right, under the flat part of the roof, and the class's measurement of it, 2.8 trillion operations a second, sits below the peak.

3.3 The T4 reached a third of its peak

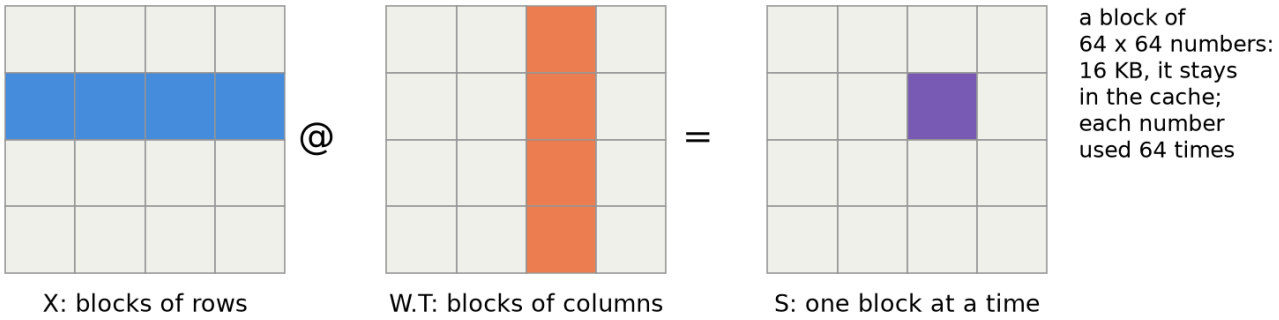
The T4	Operations a second	The product at size 4,096
its data sheet: the peak	8.1 trillion	0.017 s
published benchmarks of cuBLAS on Colab	4.6 trillion	0.030 s
the class's posts, the median of twelve	2.8 trillion	0.049 s

The class's product reached about one third of the T4's published float32 peak. That peak assumes all relevant arithmetic units remain busy and gives an upper bound, not a guaranteed application rate. Published cuBLAS benchmarks on Colab report about 4.6 trillion operations per second (github.com/gabmerlo/cuda-kernels; github.com/aaron-ang/cuda-sgemm). The reason for the difference is untested here. Benchmark repetition, warm-up, clock speed, and power limits can all affect measured throughput.

3.4 Computing the product in blocks

Optimized matrix multiplication works on blocks that can be reused in fast memory. For example, two 64 × 64 float32 blocks occupy 16 KB each. Multiplying them contributes to a 64 × 64 output block and reuses each input entry 64 times within that block product. The implementation accumulates such contributions over the

shared matrix dimension. The arithmetic count remains about $2nKm$, while data reuse reduces main-memory traffic. The full matrices can therefore exceed cache capacity without losing the reuse available within each block.



The product in blocks: one block of rows of X and one block of columns of W.T make one block of S, and both stay in the cache while they are used.

3.5 When copying takes longer than computing

In these GPU examples, the inputs must be transferred to GPU memory before computation. The T4's PCIe link is rated at up to 16 GB/s in each direction. Dividing bytes by this rate gives an ideal transfer-time lower bound. The table compares that estimate with the measured matrix-product time and the bandwidth-based lower bound for array addition.

OUTPUT

```
computation                              copies across the link      GPU compute time
X @ W.T at size 4,096 on the T4               12.6 milliseconds     48.7 milliseconds
a + b on 16.8 million numbers on the T4       12.6 milliseconds    0.671 milliseconds
```

The two inputs and output of the 4,096-square product total about 201 MB. Their estimated transfer time is at least 12.6 milliseconds, compared with 48.7 milliseconds for computation. The same transfer volume applies to addition of two arrays with 16.8 million entries, but that addition has an ideal GPU time below a millisecond. Transfers can therefore dominate a short GPU operation. Keeping data on the GPU across multiple operations amortizes this cost.

The code below measures addition with arrays already on the T4. The roofline bandwidth bound predicts at most 25 billion additions per second, corresponding to 300 GB/s of useful array traffic.

```
n = 2 ** 26                                          # 67 million numbers in each array
a = torch.rand(n, device="cuda")
b = torch.rand(n, device="cuda")
c = torch.empty(n, device="cuda")
torch.add(a, b, out=c); torch.cuda.synchronize()     # once, not measured
start = time.perf_counter()
for _ in range(20):
    torch.add(a, b, out=c)
torch.cuda.synchronize()
seconds = (time.perf_counter() - start) / 20
print(f"{n / seconds / 1e9:.1f} billion additions a second, {12 * n / seconds / 1e9:.0f}
GB/s")
```

Using peak rates gives an ideal lower bound on running time:

$$\text{time} \geq \max\left(\frac{\text{operations}}{\text{peak rate}}, \frac{\text{bytes moved}}{\text{bandwidth}}\right).$$

Actual time can be longer because of overhead and incomplete use of the hardware. The bandwidth must correspond to the memory level supplying the data. Big O describes growth in the operation count; this bound additionally accounts for data movement and hardware rates.

A deeper look 2. What computing costs

4.1 Running time, energy, and rental cost

$$\text{time} = \frac{\text{operations}}{\text{operations per second}}, \qquad \text{energy} = \text{power} \times \text{time}, \qquad \text{money} = \text{price per hour} \times \text{hours}.$$

Power is measured in **watts**, or joules per second. Multiplying average power by elapsed time gives energy. A **kilowatt-hour** is 3.6 million joules. Published power limits can support upper-bound estimates when used as limits; actual consumption requires a power or energy measurement during the run.

Processor	Power	Source
Colab's T4	70 W at most	NVIDIA's data sheet
the CPU of the machine of these notes, an Intel Core i9-14900HX	55 W at its base speed, 157 W at most	Intel
a V100 training GPT-3	330 W, with its share of the server	measured; Patterson et al., 2021

Colab does not publish the model or the power of its CPUs.

4.2 Energy estimate for one matrix product

OUTPUT

```
X @ W.T at size 4,096: 137 billion operations
processor                              seconds  watts   joules   operations a joule
CPU of this machine, 32 logical cores    0.187    157     29.3         4.69 billion
Colab's T4: your posts                   0.049     70      3.4         40.3 billion
```

The CPU row is this machine's own time for the product of Part 2, at the 157 watts it may draw at most; the T4 row is the class's median time at the T4's 70 watts. Both are upper bounds on the energy. At these rated-power estimates, the GPU has more operations per joule: 40.3 billion against 4.69 billion. These estimates do not establish the ratio of actual energy consumption. That would require measuring power or energy during the runs. At data-centre scale, both processor time and energy contribute to cost.

4.3 GPU rental prices and workload cost

	Price	Source
a T4, rented by the hour	$0.35	Google Cloud, on demand, Iowa region, the GPU alone
a V100, rented by the hour	$2.48	Google Cloud, on demand, the same
Colab's T4	free, when one is spare	Google Colab
electricity, a kilowatt-hour	15 cents	U.S. average over all sectors, July 2026, EIA

The quoted cloud prices cover the GPU alone; the virtual machine's CPU, memory, and disk cost extra. At 70 W, a T4 uses 0.07 kWh per hour, approximately one cent of electricity at the quoted rate. Rental charges also cover hardware, infrastructure, cooling, and service. The following table estimates GPU rental charges in cents:

OUTPUT

```
on a rented T4, at $0.35 an hour, in cents          rent     electricity
one hour                                              35               1
one product at size 4,096                        0.00047        0.000014
a trillion operations, at your rate               0.0034          0.0001
the layer of the notebook: 94 billion             0.00032       0.0000097
```

At the quoted price, one second of T4 rental costs about a hundredth of a cent. The measured product takes 0.049 seconds, and a trillion operations at the class's rate cost about 0.0034 cents of GPU rental. These calculations use the stated benchmark rate and price.

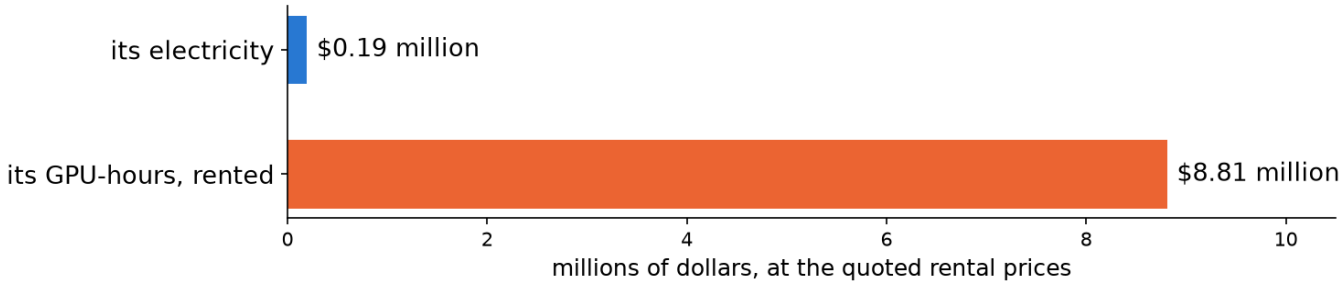
4.4 A model trained at scale: GPT-3

GPT-3, the language model of 2020	
operations to train it	3.14×10^{23}
processors	10,000 V100 GPUs
time	14.8 days
power of each, with its share of the server	330 W
energy	1,287 MWh
carbon dioxide, and its equivalents	552 tonnes

OpenAI reported 3.14×10^{23} operations for training the 175-billion-parameter GPT-3 model. Patterson and colleagues estimated its hardware use, energy, and emissions from published OpenAI and NVIDIA figures: 10,000 V100 GPUs for about two weeks, 1,287 MWh, and 552 tonnes of carbon-dioxide-equivalent emissions. The calculations below apply the rates and prices used in these notes.

OUTPUT

```
GPT-3: 3.14 x 10^23 operations
on one T4 at your rate:              3,526 years
on 10,000 V100s, as it was trained:   14.8 days, 3.55 million GPU-hours
its electricity, 1,287 MWh, at today's U.S. average price:  $192,921
its GPU-hours at the quoted list price for a V100:          $8,808,960
```



Estimated electricity and GPU rental costs using the prices quoted in these notes. These are not reported payments for the original training run.

Extrapolating the class's T4 rate gives about 3,526 years for the reported operation count. At the quoted prices, the estimated electricity use would cost about $192,921, and the estimated V100 GPU-hours would cost about $8,808,960 to rent. The rental estimate is substantially larger than the electricity estimate. These illustrate costs at the stated prices; they do not establish what OpenAI paid in 2020.

4.5 Estimating cost from throughput and price

$$\text{cost} \approx \text{count} \times \text{seconds for one operation} \times \text{price of a second}.$$

The operation count is independent of the machine. O( ) describes its growth with input size. Time per operation depends on the implementation, access order, cache, and processor. Power and hourly rental price convert elapsed time into energy and monetary estimates. Compare achieved throughput and hourly price for the workload when choosing a processor.

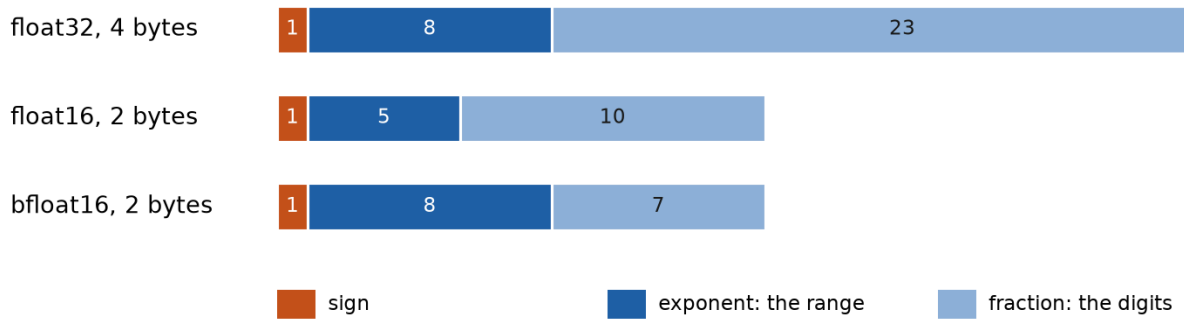
Hardware and software for matrix multiplication

Matrix multiplication is a major workload in machine learning. The examples below show how hardware and software are designed for it. The figures come from published data sheets and papers, listed at the end of the section.

5.1 T4 tensor cores and reduced precision

The T4 multiplies	With	Operations a second
32-bit floats: what the class measured	2,560 CUDA cores	8.1 trillion
16-bit floats, with 32-bit sums	320 tensor cores	65 trillion
8-bit integers	320 tensor cores	130 trillion
4-bit integers	320 tensor cores	260 trillion

Besides its 2,560 ordinary cores, the T4 has 320 **tensor cores**, specialized units for multiplying small matrices of 16-bit or smaller numbers. In 16-bit numbers with 32-bit sums the data sheet gives 65 trillion operations a second, eight times the 32-bit peak; in 8-bit and 4-bit integers, twice and four times that again. The class's product used float32 and the ordinary cores, and reached 2.8 trillion; the tensor cores' 16-bit peak is more than twenty times that rate, on the same chip.



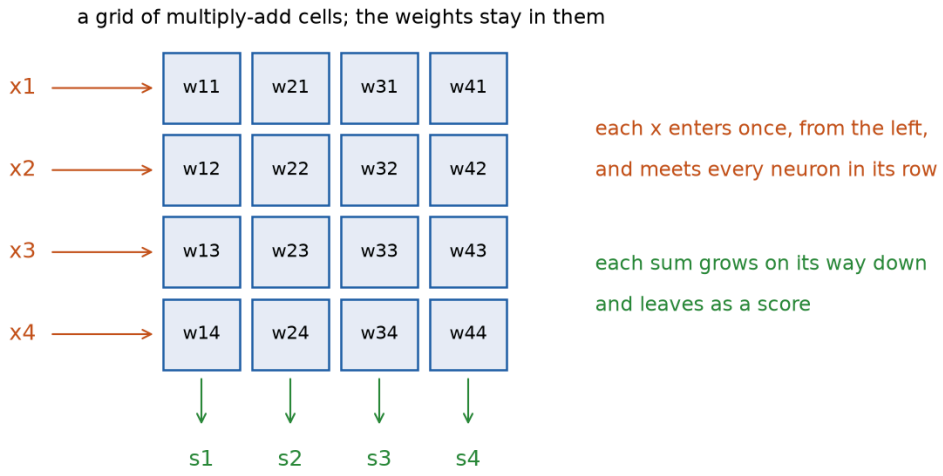
Three formats of a number: the sign, the exponent that sets the range, and the fraction that sets the digits.

Float32 uses one sign bit, 8 exponent bits, and 23 stored fraction bits. Float16 uses 5 exponent bits and 10 fraction bits. **bfloat16** uses 8 exponent bits and 7 fraction bits, retaining a wide range with lower precision. Many neural-network computations can use such formats, although the required precision depends on the computation.

Smaller formats reduce memory traffic for the same number of values and can make arithmetic hardware smaller. In the cited energy estimates, a 16-bit floating-point multiplication requires 1.1 picojoules compared with 3.7 for 32 bits. Google's cited bfloat16 description reports a multiplier about eight times smaller than a float32 multiplier.

5.2 The TPU and systolic arrays

In 2017 Google described its Tensor Processing Unit, a chip specialized for neural-network inference. Its matrix unit has 65,536 multiply-add cells for 8-bit numbers, 92 trillion operations a second at its peak, and 28 mebibytes of memory on the chip itself, managed by the software. On Google's own neural networks it was on average 15 to 30 times as fast as the GPU and the CPU of its day, and did 30 to 80 times as many operations for each watt. Its architecture dedicates much of its hardware to matrix arithmetic.

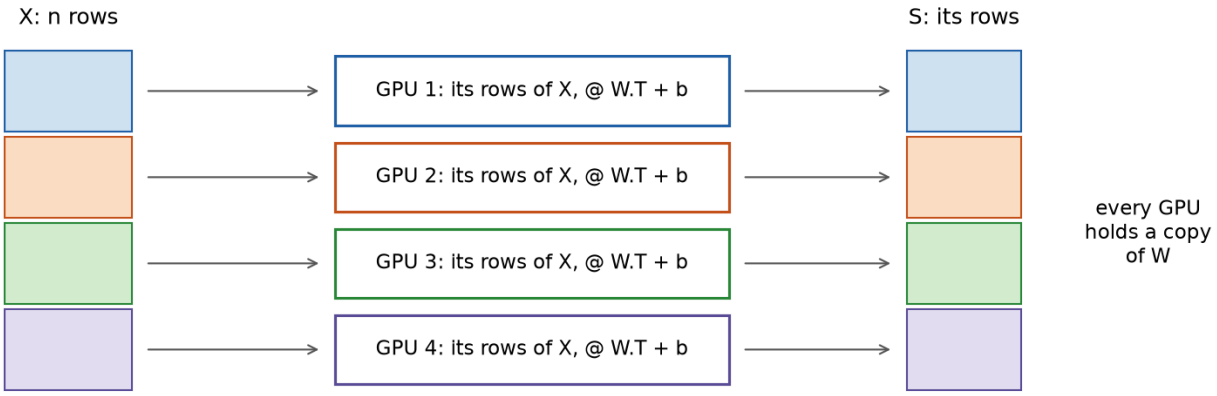


A systolic array: the weights stay in the cells, the coordinates of a point flow through, and the scores leave at the bottom.

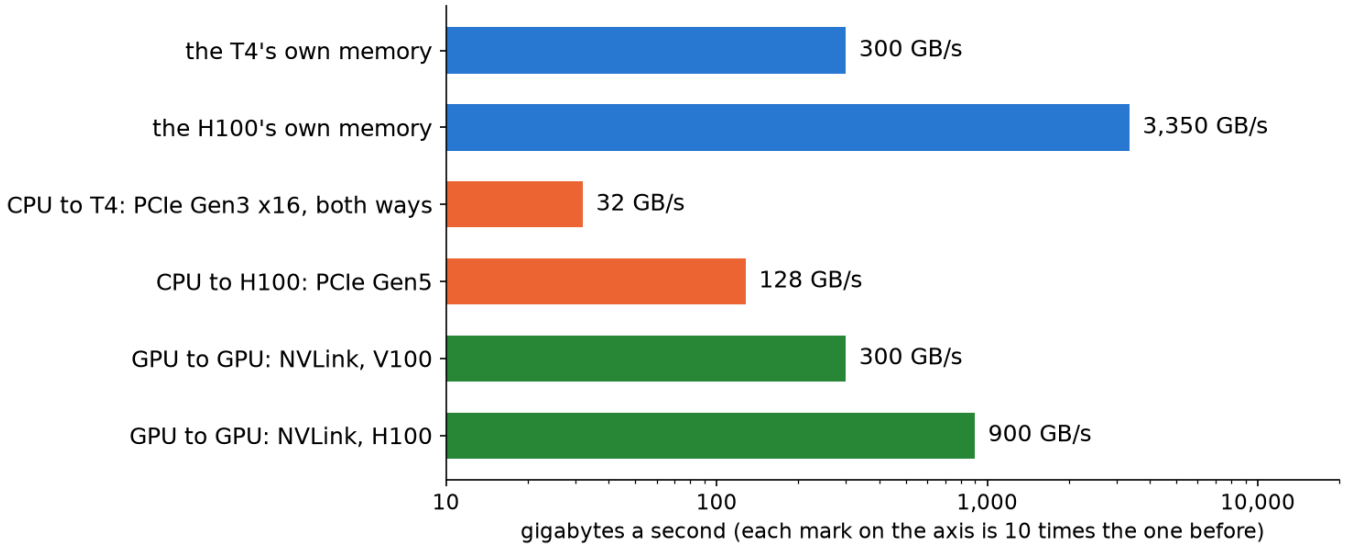
A **systolic array** arranges multiply-add cells in a grid. In this diagram, each cell holds a weight. Input coordinates pass along rows, while partial sums pass down columns. Each cell multiplies the incoming coordinate by its weight and adds the result to the partial sum. Reusing values across neighbouring cells reduces repeated memory transfers. The resulting arithmetic intensity also depends on precision and the traffic needed for weights and outputs.

5.3 Distributing work across GPUs

Large training workloads distribute computation across GPUs. In the illustrated arrangement, each GPU receives a subset of the input rows and uses its copy of `W` to compute the corresponding output rows. During training, the GPUs combine their gradient contributions and synchronize the weights before the next step. Communication therefore contributes to running time.



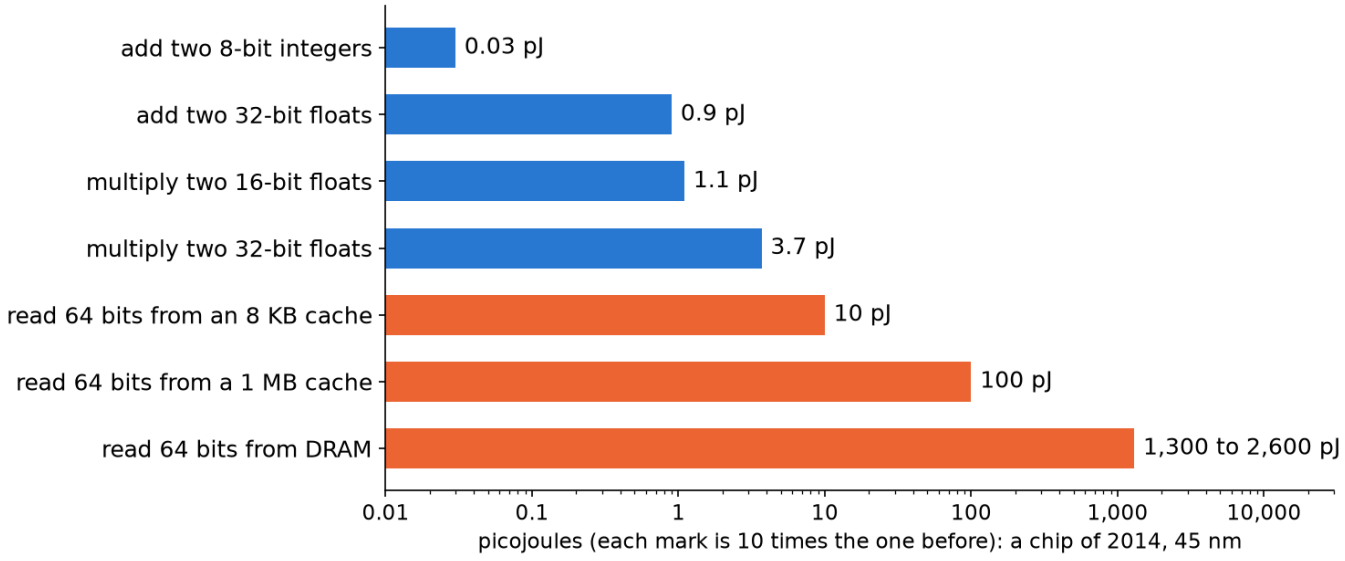
The rows of X split over four GPUs, each with a copy of W: each computes its rows of S.



How many bytes a second each link delivers, by the data sheets of the T4, the V100 and the H100.

A GPU reads its own memory fastest: 300 gigabytes a second on the T4, 3,350 on an H100, the chip of 2022. The PCIe and NVLink figures in the chart are aggregate bidirectional rates, whereas the memory figures follow the data sheets' memory-bandwidth convention; they are not one-way copy rates. To join GPUs to each other NVIDIA built a link of its own, **NVLink**: 300 gigabytes a second for a V100, 900 for an H100, and its switch system joins up to 256 H100s. Between servers the network is narrower still. Large training systems must account for these bandwidth differences when distributing work and data.

5.4 Energy costs of arithmetic and memory access



Arithmetic and memory-access energy estimates for 45-nm hardware, from Horowitz (ISSCC 2014).

Horowitz's 2014 estimates for 45-nm hardware compare arithmetic with memory access. A 32-bit floating-point multiplication uses about 3.7 picojoules. A 64-bit read costs about 10 picojoules from a small cache, 100 from a larger cache, and 1,300–2,600 from DRAM. These historical estimates illustrate how data movement

can dominate arithmetic energy; actual values depend on the hardware. Keeping frequently reused data near the arithmetic units can reduce those transfers.

5.5 Software and hardware designed for each other

Optimized matrix libraries. NumPy typically delegates matrix multiplication to a **BLAS** implementation, such as OpenBLAS or Intel MKL. PyTorch can use cuBLAS on NVIDIA GPUs. These libraries tune block sizes, vector instructions, and thread scheduling to the processor. Such implementation choices help explain the large throughput difference between Python loops and compiled matrix products with the same asymptotic arithmetic count.

FlashAttention. An attention computation forms scores QK^T, applies a row-wise softmax, and multiplies the result by V. Storing and rereading the full score matrix can require substantial GPU-memory traffic for long sequences. FlashAttention (Dao and colleagues, 2022) computes attention in blocks using fast on-chip memory, avoiding storage of the full score matrix. It retains the same asymptotic arithmetic complexity. During the backward pass it recomputes intermediate values, trading extra arithmetic for fewer memory transfers. The paper reports a threefold GPT-2 training speedup in its benchmark.

Numerical formats and structured sparsity. Google's TPUs helped motivate bfloat16. NVIDIA sparse tensor cores support **2:4 sparsity**, in which two entries in each supported group of four are zero. Compatible models and training methods can use that pattern to reduce multiplication work while seeking to preserve accuracy. The cited H100 specification gives 1,979 trillion 16-bit operations per second with sparsity, twice its corresponding dense peak.

Wafer-scale hardware. Most processors are individual dies cut from a silicon wafer. Cerebras uses a wafer-scale design for its WSE-3. Its product page specifies 46,225 square millimetres, 4 trillion transistors, and 900,000 cores. Memory is distributed alongside the cores to reduce off-chip transfers; the transfers required by a particular workload still depend on its size and mapping.

5.6 Operation counts and hardware design

The layer computation uses O(nKm) arithmetic operations. Its running time also depends on interpreter overhead, memory layout, caches, parallel execution, communication, numerical precision, and scheduling. Operation counts and hardware measurements together explain the performance differences seen in this meeting.

Sources of the figures in this section.

- NVIDIA, the data sheets of the T4 (2,560 CUDA cores, 320 tensor cores; 8.1, 65, 130 and 260 trillion operations a second; 16 GB at 300 GB/s; PCIe 32 GB/s; 70 W), the V100 (NVLink 300 GB/s) and the H100 (memory 3.35 TB/s, NVLink 900 GB/s, PCIe 128 GB/s, 1,979 TFLOPS with sparsity, up to 256 GPUs in one switch system).
- N. P. Jouppi et al., *In-Datacenter Performance Analysis of a Tensor Processing Unit*, ISCA 2017.
- M. Horowitz, *Computing's energy problem (and what we can do about it)*, ISSCC 2014, Figure 1.1.9.
- T. Dao, D. Y. Fu, S. Ermon, A. Rudra and C. Ré, *FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness*, 2022.
- A. Mishra et al., *Accelerating Sparse Deep Neural Networks*, 2021.
- S. Wang and P. Kanwar, *BFloat16: The secret to high performance on Cloud TPUs*, Google Cloud blog, 2019.
- Cerebras, the product page of the WSE-3.
- D. Patterson et al., *Carbon Emissions and Large Neural Network Training*, 2021, for GPT-3; Google Cloud's on-demand prices and the U.S. Energy Information Administration's average price of electricity, as in A deeper look 2.

Matrix-multiplication exponents

The cubic count describes the three-loop algorithm. Other algorithms can reduce the exponent: Strassen's recursion uses seven half-size products, giving $\log_2 7 \approx 2.807$. Dupont et al. (August 2026) report $\omega < 2.371177$. The OpenAI preprint dated October 2, 2026, released October 6, reports $\omega \le 9 / 4 = 2.25$ over the complex numbers.

The stated bound is $O_\varepsilon(n^{9/4+\varepsilon})$ arithmetic operations for every fixed $\varepsilon > 0$, with constants that may depend on ε. The preprint does not specify the finite matrix size at which an implementation would become competitive. Assessing practical performance also requires constants, memory costs, and measurements.